

# Linux Performance Monitoring

## **Supervision et analyse des performances d'un système Linux**

**Auteur:** Guewen Klett VOUBOU

**Formation:** Mastère 2 ingénieur Systèmes, Réseaux et Cybersécurité

**Etablissement:** Nexa Digital School



|                                                                           |          |
|---------------------------------------------------------------------------|----------|
| <b>Présentation du projet.....</b>                                        | <b>3</b> |
| <b>Objectifs du projet.....</b>                                           | <b>3</b> |
| <b>Partie I : Analyse des performances d'un système Linux.....</b>        | <b>4</b> |
| <b>I- Analyse des performances CPU et RAM.....</b>                        | <b>4</b> |
| <b>II- Analyse approfondie de la mémoire vive (RAM).....</b>              | <b>5</b> |
| <b>III- Analyse des performances disque.....</b>                          | <b>6</b> |
| <b>IV- Analyse d'activité réseau.....</b>                                 | <b>7</b> |
| <b>PARTIE 2: Automatisation de la collecte des métriques système.....</b> | <b>8</b> |
| <b>I- Développement d'un script Bash de supervision.....</b>              | <b>8</b> |



## Présentation du projet

La supervision des performances constitue une activité essentielle de l'administration des systèmes Linux. Elle permet d'identifier rapidement les goulots d'étranglement, d'anticiper les dégradations de performances et de garantir la disponibilité des services.

Dans le cadre de ce projet, une série d'exercices a été réalisée afin d'étudier le comportement d'un système Linux face à différentes charges de travail. Les analyses ont porté sur les principaux composants du système, notamment le processeur, la mémoire vive, les entrées/sorties disque ainsi que les performances réseau. Le projet s'est ensuite prolongé par l'automatisation de la collecte des métriques à l'aide de scripts Bash et Python.

## Objectifs du projet

Les principaux objectifs de ce projet étaient les suivants :

- Comprendre les indicateurs de performances d'un système Linux.
- Analyser la consommation des ressources matérielles.
- Identifier les processus responsables d'une forte utilisation CPU ou mémoire.
- Étudier l'impact d'une charge artificielle générée avec **stress**.
- Observer l'activité disque et réseau.
- Automatiser la collecte des métriques système.
- Produire des rapports exploitables à partir des données collectées.

## **Partie I :** Analyse des performances d'un système Linux

La supervision des performances est une étape essentielle de l'administration des systèmes Linux. Elle permet d'évaluer l'état de santé d'une machine, de détecter les processus les plus consommateurs de ressources et d'anticiper les éventuelles dégradations des performances.

Dans cette première partie, plusieurs commandes Linux ont été utilisées afin d'analyser le comportement du système. Les observations portent sur quatre axes principaux :

- l'utilisation du processeur (CPU) ;
- la consommation de la mémoire vive (RAM) ;
- les performances des entrées/sorties disque (I/O) ;
- l'activité réseau.

### **I- Analyse des performances CPU et RAM**

#### **Commandes Utilisées:**

- htop ou top pour surveiller le cpu

**Htop** est un utilitaire interactif permettant de superviser un système Linux en temps réel. Contrairement à la commande top, il offre une interface plus lisible facilitant l'identification des processus actifs ainsi que l'analyse de leur consommation en ressources.

Les principaux indicateurs observés sont:

| Indicateur   | Description                                      |
|--------------|--------------------------------------------------|
| US           | Temps CPU utilisé par les processus utilisateurs |
| SY           | Temps CPU utilisé par le noyau Linux             |
| VIRT         | Mémoire virtuelle allouée au processus           |
| RES          | Mémoire physique réellement utilisée             |
| SHR          | Mémoire partagée avec d'autres processus         |
| CPU %        | Pourcentage d'utilisation du processeur          |
| MEM %        | Pourcentage d'utilisation du processeur          |
| Uptime       | Temps écoulé depuis le démarrage du système      |
| Load Average | Charge moyenne du système sur 1, 5 et 15 minutes |

```

1 [ 0.0%] Tasks: 103, 218 thr; 1 running
2 [ 0.7%] Load average: 0.07 0.07 0.03
Mem [ 1019M/3.79G] Uptime: 00:48:54
Swap [ 0K/923M]

PID USER      PRI  NI  VIRT   RES   SHR  S  CPU% MEM%   TIME+  Command
6290 klett      20   0 11264 4672 3316 R   0.7  0.1  0:14.53 http
3694 klett      20   0 301M 76376 42260 S   0.0  1.9  0:27.33 /usr/lib/xorg/Xorg vt2 -displayfd 3 -auth /run/user/1000/gdm/Xauthority -background none -noreset -keeptty -verbose
1851 root         20   0 233M 7480 6472 S   0.0  0.2  0:05.04 /usr/bin/vmtoolsd
4478 klett      20   0 803M 55540 40540 S   0.0  1.4  0:12.05 /usr/libexec/gnome-terminal-server
3899 klett      20   0 3891M 248M 101M S   0.0  6.4  0:42.66 /usr/bin/gnome-shell
3762 klett      20   0 301M 76376 42260 S   0.0  1.9  0:03.06 /usr/lib/xorg/Xorg vt2 -displayfd 3 -auth /run/user/1000/gdm/Xauthority -background none -noreset -keeptty -verbose
4103 klett      20   0 141M 41392 29376 S   0.0  1.0  0:04.65 /usr/bin/vmtoolsd -n vmusr --blockFd 3
3624 root         20   0 307M 10120 8384 S   0.0  0.3  0:00.06 gdm-session-worker [pam/gdm-password]
3913 klett      20   0 3891M 248M 101M S   0.0  6.4  0:00.71 /usr/bin/gnome-shell
4017 klett      20   0 556M 30968 20704 S   0.0  0.8  0:00.43 /usr/libexec/gsd-color
3531 colord      20   0 243M 16404 11116 S   0.0  0.4  0:00.14 /usr/libexec/colord
4060 klett      20   0 556M 30968 20704 S   0.0  0.8  0:00.03 /usr/libexec/gsd-color
4091 klett      20   0 340M 31108 20828 S   0.0  0.8  0:00.03 /usr/libexec/gsd-power
4091 klett      20   0 340M 31388 20660 S   0.0  0.8  0:00.35 /usr/libexec/gsd-xsettings
3635 klett      20   0 23412 13972 8160 S   0.0  0.4  0:00.70 /lib/systemd/systemd --user
3650 klett      20   0 11312 8584 3948 S   0.0  0.2  0:00.99 /usr/bin/dbus-daemon --session --address=systemd: --nofork --nopidfile --systemd-activation --syslog-only
3656 klett      20   0 234M 7668 6716 S   0.0  0.2  0:00.04 /usr/libexec/gvfsd
738 messagebu 20   0 9848 6184 3812 S   0.0  0.2  0:01.86 /usr/bin/dbus-daemon --system --address=systemd: --nofork --nopidfile --systemd-activation --syslog-only
775 root         20   0 17424 8100 7020 S   0.0  0.2  0:00.38 /lib/systemd/systemd-logind
3654 klett      20   0 234M 7668 6716 S   0.0  0.2  0:00.09 /usr/libexec/gvfsd
4100 klett      20   0 774M 59100 43224 S   0.0  1.5  0:00.41 /usr/libexec/evolution-data-server/evolution-alarm-notify
358 root         19  -1 52640 19712 17796 S   0.0  0.5  0:00.82 /lib/systemd/systemd-journald
3967 klett      20   0 963M 31024 26668 S   0.0  0.8  0:00.19 /usr/libexec/evolution-calendar-factory
3944 klett      20   0 567M 20164 17520 S   0.0  0.5  0:00.13 /usr/libexec/gnome-shell-calendar-server
3925 klett      20   0 271M 33320 19640 S   0.0  0.8  0:01.48 /usr/libexec/libus-extension-gtk3
4002 klett      20   0 310M 10724 9400 S   0.0  0.3  0:00.05 /usr/libexec/gvfsd-trash --spawner :1.3 /org/gtk/gvfs/exec_spawn/0
3883 klett      20   0 611M 17100 14324 S   0.0  0.4  0:00.19 /usr/libexec/gnome-session-binary --systemd-service --session=ubuntu
3653 klett      39  19 1091M 24208 10108 S   0.0  0.6  0:00.03 /usr/libexec/tracker-nlrunner-fs
3951 klett      20   0 567M 20164 17520 S   0.0  0.5  0:00.02 /usr/libexec/gnome-shell-calendar-server
3977 klett      20   0 158M 6368 5716 S   0.0  0.2  0:00.13 /usr/libexec/gvfsd-metadata

```

## Analyse des résultats:

- 1-Le processus avec le PID **6290** correspond à la commande lancées durant la manipulation.
- 2- Le cœur du processus le plus sollicité est **0,7**.

## II- Analyse approfondie de la mémoire vive (RAM)

### Commandes Utilisées:

- free
- vmstat
- stress

### Analyse des résultats:

Après l'installation de stress et son lancement avec commande **sudo stress --cpu 1 --vm 1 --vm-bytes 2G** grâce à la la commande **free -h** nous constatons que la quantité de mémoire utilisée augmente jusqu'à atteindre **2,4 GiB**. Cette augmentation est liée à l'allocation de mémoire supplémentaire par le système pour exécuter la charge demandée.

Les indicateurs que nous observons sont :

- **swap**: mémoire virtuelle utilisée ;
- **free**: mémoire physique libre;
- **buff/cache**: mémoire utilisée pour les buffers;
- **available**: Mémoire disponible

```

klett@klett-VMware-Virtual-Platform:~$ free -h
              total        utilisé        libre        partagé tamper/cache        dispo
Mem:          3,8Gi          2,7Gi          522Mi          37Mi          865Mi          1
,1Gi
Échange:       3,6Gi          210Mi          3,4Gi

```

- **iostat -x 1** : Pour surveiller le réseau
- **dd if=/dev/zero of=testfile.img bs=2M count=4096 status=progress**: Pour lancer une écriture de disque intensive
- **iostat**: Affiche quelle application lit ou écrit dans le disque.

**Analyse des résultats :**

Lors du lancement de la commande **dd** un fichier de taille 5,6 à été crée afin de provoquer une activité importante sur le disque. Les opérations se sont déroulés sur la partition **sda**

```
Device      r/s    kB/s    rrqm/s  %rrqm  r_await  rareq-sz  w/s    kB/s    wrqm/s  %drqm  d_await  da
/s  wrqm/s  %wrqm  w_await  wareq-sz  d/s    kB/s    drqm/s  %drqm  d_await  da
req-sz    f/s  f_await  aqu-sz  %util
loop0      0,00    0,00    0,00    0,00    0,00    0,00    0,00    0,00    0,00    0,00    0,
0,00    0,00    0,00    0,00    0,00
loop1      0,00    0,00    0,00    0,00    0,00    0,00    0,00    0,00    0,00    0,00    0,
0,00    0,00    0,00    0,00    0,00
loop10     0,00    0,00    0,00    0,00    0,00    0,00    0,00    0,00    0,00    0,00    0,
0,00    0,00    0,00    0,00    0,00
loop11     0,00    0,00    0,00    0,00    0,00    0,00    0,00    0,00    0,00    0,00    0,
0,00    0,00    0,00    0,00    0,00
loop12     0,00    0,00    0,00    0,00    0,00    0,00    0,00    0,00    0,00    0,00    0,
0,00    0,00    0,00    0,00    0,00
loop13     0,00    0,00    0,00    0,00    0,00    0,00    0,00    0,00    0,00    0,00    0,
0,00    0,00    0,00    0,00    0,00
loop14     0,00    0,00    0,00    0,00    0,00    0,00    0,00    0,00    0,00    0,00    0,
0,00    0,00    0,00    0,00    0,00
```

```
0,00    0,00    0,00    0,00    0,00
loop5      0,00    0,00    0,00    0,00    0,00    0,00    0,00    0,00    0,00    0,00    0,
0,00    0,00    0,00    0,00    0,00
loop6      0,00    0,00    0,00    0,00    0,00    0,00    0,00    0,00    0,00    0,00    0,
0,00    0,00    0,00    0,00    0,00
loop7      0,00    0,00    0,00    0,00    0,00    0,00    0,00    0,00    0,00    0,00    0,
0,00    0,00    0,00    0,00    0,00
loop8      0,00    0,00    0,00    0,00    0,00    0,00    0,00    0,00    0,00    0,00    0,
0,00    0,00    0,00    0,00    0,00
loop9      0,00    0,00    0,00    0,00    0,00    0,00    0,00    0,00    0,00    0,00    0,
0,00    0,00    0,00    0,00    0,00
sda      48,00  96,00    0,50  100,00    0,00    0,00    0,00    2,00  200,00
sr0      0,00    0,00    0,00    0,00    0,00    0,00    0,00    0,00    0,00    0,00    0,
0,00    0,00    0,00    0,00    0,00
sr1      0,00    0,00    0,00    0,00    0,00    0,00    0,00    0,00    0,00    0,00    0,
0,00    0,00    0,00    0,00    0,00
```

```
avg-cpu:  %user   %nice %system %iowait  %steal   %idle
           5,56    0,00    3,54    0,00    0,00   90,91
```

| Indicateur | Description                          |
|------------|--------------------------------------|
| %util      | Pourcentage d'occupation du disque   |
| await      | Temps moyen d'attente des opérations |
| wKB/s      | Débit moyen d'écriture en Mo/s       |

## IV- Analyse d'activité réseau

### Commandes utilisées:

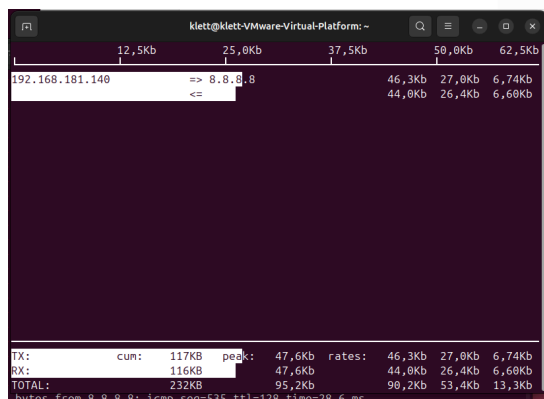
- **iftop** : Pour surveiller le réseau
- **ping -i 0.01 -c 1000 8.8.8.8**: Pour lancer un ping flood

### Analyse des résultats:

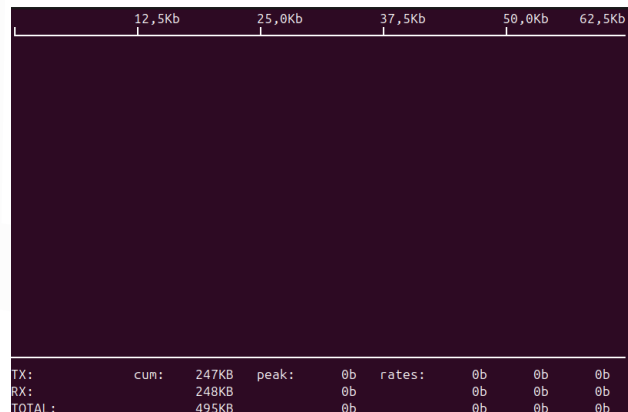
Suite à notre surveillance réseau après avoir lancé le ping flood sollicite l'adresse **8.8.8.8** pendant l'échange réseau. Les débits évoluent au cours du test avant de revenir au niveau stable.

Cette analyse nous permet de vérifier que les communications réseau sont correctement établies et d'identifier rapidement une éventuelle saturation du lien réseau.

### Débit sortant durant le test



### débit sortant à la fin du test



## **PARTIE 2: Automatisation de la collecte des métriques système**

Après l'analyse manuelle des performances système réalisée dans la première partie, cette seconde phase du projet a pour objectif d'automatiser la collecte des principales métriques système. L'automatisation permet de surveiller en continu les ressources d'un serveur Linux sans intervention humaine et constitue une pratique courante dans les environnements de production.

Deux approches ont été étudiées :

- une première implémentation basée sur des scripts **Bash** utilisant les commandes natives de Linux ;
- une seconde implémentation développée en **Python** grâce à la bibliothèque **psutil**, permettant une collecte plus évoluée et plus facilement exploitable.

Les métriques collectées sont ensuite exportées dans un fichier CSV afin de faciliter leur exploitation, leur archivage ou leur visualisation graphique.

### **I- Développement d'un script Bash de supervision**

**Environnements utilisés:**

**Ubuntu**

**Bash**

**Visual studio Code**

**Outils:** top, free, iostat, ifstat, awk, grep

#### **1- Collecte des informations processeur**

**Script:**

```
klett@ubuntu:~/scripting$ top -bn1 | grep "Cpu(s)" | awk -F',' '{print $1}' | awk '{print $2}'  
33.3
```

La commande récupérera l'utilisation du processeur, de la mémoire, du disque ainsi que du réseau.

#### **2- Collecte des information mémoire**

**Scrip:**

- Mémoire Utilisée

```
klett@ubuntu:~/scripting$ free -m | grep '^Mem:' | awk '{print $3}'  
2334
```



- Mémoire libre

```
klett@ubuntu:~/scripting$ free -m | grep '^Mem:' | awk '{print $4}'
124
```

### 3- Collecte des performances du disque

Afin de mesurer les performances d'entrées et de sorties, on utilise la commande **iostat**.

Les informations que nous avons récupérées sont le **disk read** et le **disk write**. Ces valeurs nous permettent d'évaluer l'activité du périphérique de stockage.

```
klett@ubuntu:~/scripting$ iostat -k 1 1 | grep '^sda' | awk '{print $3}'
170.07
klett@ubuntu:~/scripting$ iostat -k 1 1 | grep '^sda' | awk '{print $4}'
3313.09
```

### 4- Collecte des métriques réseau

Les statistiques réseau sont obtenues grâce à la commande **ifstat**.

Script:

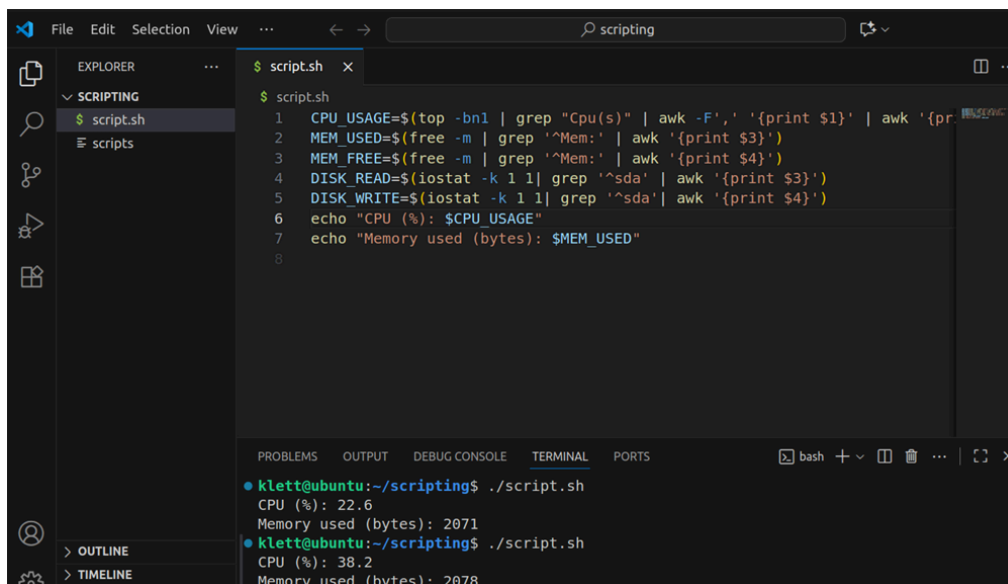
```
klett@ubuntu:~/scripting$ ifstat -i ens33 1 1 | tail -n 1 | awk '{print $1}'
0.71
klett@ubuntu:~/scripting$ ifstat -i ens33 1 1 | tail -n 1 | awk '{print $2}'
0.00
```

### 5- Développement du script [monitor.sh](#)

Les différentes commandes précédentes sont ensuite regroupées dans un unique script Bash nommé [monitor.sh](#) qui sera réalisé dans **virtual studio code**.

Ce script nous permettra d'automatiser la récupération des métriques et d'afficher les résultats directement dans le terminal.

Les variables utilisées correspondent aux différents indicateurs collectés tels que CPU Usage, Memory Used, Memory Free, Disk Read...



```
File Edit Selection View ... scripting
EXPLORER
  SCRIPTING
    script.sh
    scripts
  PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
$ script.sh
$ script.sh
1 CPU_USAGE=$(top -bn1 | grep "Cpu(s)" | awk -F',' '{print $1}' | awk '{pr
2 MEM_USED=$(free -m | grep '^Mem:' | awk '{print $3}')
3 MEM_FREE=$(free -m | grep '^Mem:' | awk '{print $4}')
4 DISK_READ=$(iostat -k 1 1 | grep '^sda' | awk '{print $3}')
5 DISK_WRITE=$(iostat -k 1 1 | grep '^sda' | awk '{print $4}')
6 echo "CPU (%): $CPU_USAGE"
7 echo "Memory used (bytes): $MEM_USED"
8
klett@ubuntu:~/scripting$ ./script.sh
CPU (%): 22.6
Memory used (bytes): 2071
klett@ubuntu:~/scripting$ ./script.sh
CPU (%): 38.2
Memory used (bytes): 2078
```

## 6- Automatisation de la collecte

Afin d'assurer une supervision continue, le script a il sera modifié de la sorte

**while :**

**do**

**# collect everything**

**# echo...**

**sleep 5**

**done**

pour exécuter automatiquement les mesures toutes les cinq secondes.

## 7- Export des métriques au format CSV

Toujours dans vscode

```
$ script.sh
1  OUTPUT_FILE="metrics_$(date +%Y%m%d_%H%M%S).csv"
2  echo "Timestamp,CPU_Usage,Mem_Used,Mem_Free,Disk_Read,Disk_Write,Net_Rx,Net_Tx" >
3  while :
4  do
5  TIMESTAMP=$(date +"%Y-%m-%d %H:%M:%S")
6  CPU_USAGE=$(top -bn1 | grep "Cpu(s)" | awk -F',' '{print $1}' | awk '{print $2}')
7  MEM_USED=$(free -m | grep '^Mem:' | awk '{print $3}')
8  MEM_FREE=$(free -m | grep '^Mem:' | awk '{print $4}')
9  DISK_READ=$(iostat -k 1 1 | grep '^sda' | awk '{print $3}')
10 DISK_WRITE=$(iostat -k 1 1 | grep '^sda' | awk '{print $4}')
11 NET_RX=$(ifstat -i ens33 1 1 | tail -n 1 | awk '{print $1}')
12 NET_TX=$(ifstat -i ens33 1 1 | tail -n 1 | awk '{print $2}')
13
14     echo "$TIMESTAMP,$CPU_USAGE,$MEM_USED,$MEM_FREE,$DISK_READ,$DISK_WRITE,$NET_RX,
15 >> "$OUTPUT_FILE"
16     sleep 5
17 done
```

Et on obtiendra comme fichier csv:

```
metrics_20251005_211147.csv > data
1  Timestamp,CPU_Usage,Mem_Used,Mem_Free,Disk_Read,Disk_Write,Net_Rx,Net_Tx
2  2025-10-05 21:11:47,9.1,2123,199,237.44,2171.46,11.15,0.00
3  2025-10-05 21:11:54,0.0,2126,198,237.37,2171.25,0.00,0.00
4  2025-10-05 21:12:01,2.9,2126,198,237.31,2170.65,0.00,0.15
5  2025-10-05 21:12:09,0.0,2126,198,237.24,2170.04,0.00,0.00
6  2025-10-05 21:12:16,2.9,2126,198,237.18,2169.44,0.06,0.00
7  2025-10-05 21:12:23,3.0,2126,198,237.11,2168.92,0.00,0.00
8  2025-10-05 21:12:30,0.0,2126,198,237.04,2168.32,0.06,0.00
9  2025-10-05 21:12:37,2.9,2107,214,236.98,2167.72,0.00,0.00
10 2025-10-05 21:12:45,0.0,2108,213,236.91,2167.12,0.00,0.00
11 2025-10-05 21:12:52,3.1,2110,212,236.85,2166.52,0.00,0.00
12 2025-10-05 21:12:59,2.9,2111,212,236.78,2165.92,0.00,0.16
13 2025-10-05 21:13:06,0.0,2110,212,236.71,2165.32,71.63,0.00
14 2025-10-05 21:13:14,2.9,2111,212,236.65,2164.71,0.99,0.00
15 2025-10-05 21:13:21,0.0,2111,212,236.58,2164.11,0.06,0.00
16 2025-10-05 21:13:28,0.0,2111,212,236.52,2163.52,0.00,0.00
17 2025-10-05 21:13:35,2.9,2111,212,236.45,2162.92,0.00,0.00
18 2025-10-05 21:13:42,0.0,2111,212,236.39,2162.32,0.00,0.19
```

Ce format nous facilitera l'exploitation dans excel ou tout autre outil de visualisation.